

AI FitTrack Project Design

Phase 3

This design specification documents the actual architecture, data model, API contracts, request sequence, security controls and Gemini integration implemented in the uploaded backend.

Field	Value
Project	AI FitTrack
Submission	SmartBridge Naan Mudhalvan
Team ID	SWTID-2026-8100
Team size	4
Team Leader	Parthiban M
Team Members	Naveen J; Nidesh N S; Nithish Kumar P
Project scope	Backend-only REST API
Technology	Node.js, Express.js, MongoDB, Mongoose, JWT and Google Gemini AI
Source baseline	Latest uploaded backend ZIP and supplied verification evidence
Document status	Submission-ready technical content; individual academic IDs and final links remain pending

Prepared for academic evaluation. Secrets, credentials, private environment files and unverified claims are excluded.

Document contents

1. Architecture
2. Component design
3. Database design
4. API design
5. Security and AI design
6. Error and configuration design

Architecture overview

The project uses an MVC-inspired layered backend. Express routes receive requests, middleware verifies identity, validation utilities normalize inputs, controllers coordinate use cases, Mongoose models persist data and service modules encapsulate JWT and Gemini behavior. The REST response is the view returned to the API consumer.

Implemented backend architecture

Request path and service boundaries in the uploaded Node.js project

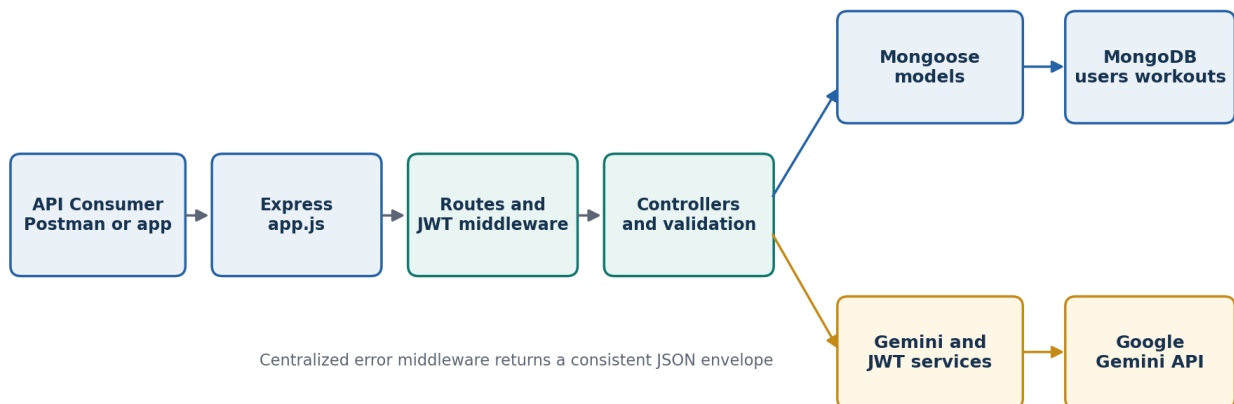


Figure 1 Implemented backend components and external dependencies

Component responsibilities

Layer	Files	Responsibility
Gateway	app.js, server.js	Load configuration, CORS and body parsing; connect database; mount routes; start and stop server safely.

Layer	Files	Responsibility
Routes	routes/*.js	Map HTTP methods and paths to controller functions; apply JWT protection to protected groups.
Middleware	middleware/*.js	Authenticate Bearer tokens, load the current user and produce centralized 404 and error responses.
Controllers	controllers/*.js	Execute authentication, workout and AI use cases and return standard responses.
Models	models/User.js, models/Workout.js	Define document shape, validation, hooks, indexes and safe serialization.
Services	services/jwtService.js, geminiService.js	Sign and verify JWTs; generate structured Gemini content with timeout and retry handling.
Utilities	utils/*.js	Input validation, safe async handling, API errors and response helpers.
Verification	tests/, postman/	Automated behavior checks and ordered manual request collection.

Database design

MongoDB document relationship

Mongoose schemas and implemented validation boundaries

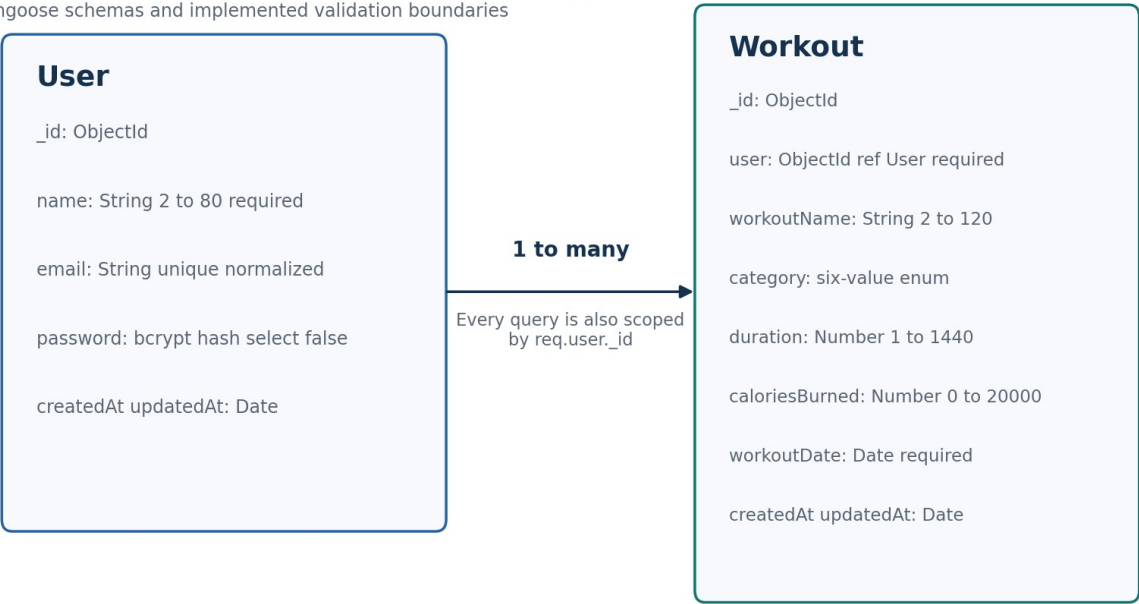


Figure 2 User to Workout one-to-many document relationship

User field	Type	Constraint
_id	ObjectId	MongoDB primary identifier
name	String	Required, trimmed, 2-80 characters
email	String	Required, unique, lowercase, validated
password	String	Required, bcrypt hash, select false
createdAt / updatedAt	Date	Automatically maintained

Workout field	Type	Constraint
_id	ObjectId	MongoDB primary identifier
user	ObjectId ref User	Required, indexed owner
workoutName	String	Required, trimmed, 2-120 characters

Workout field	Type	Constraint
category	String	Required, six-value enumeration
duration	Number	Required, 1-1440 minutes
caloriesBurned	Number	Required, 0-20000
workoutDate	Date	Required
createdAt / updatedAt	Date	Automatically maintained

Protected request design

Protected request sequence

Example for a workout create update search or delete request

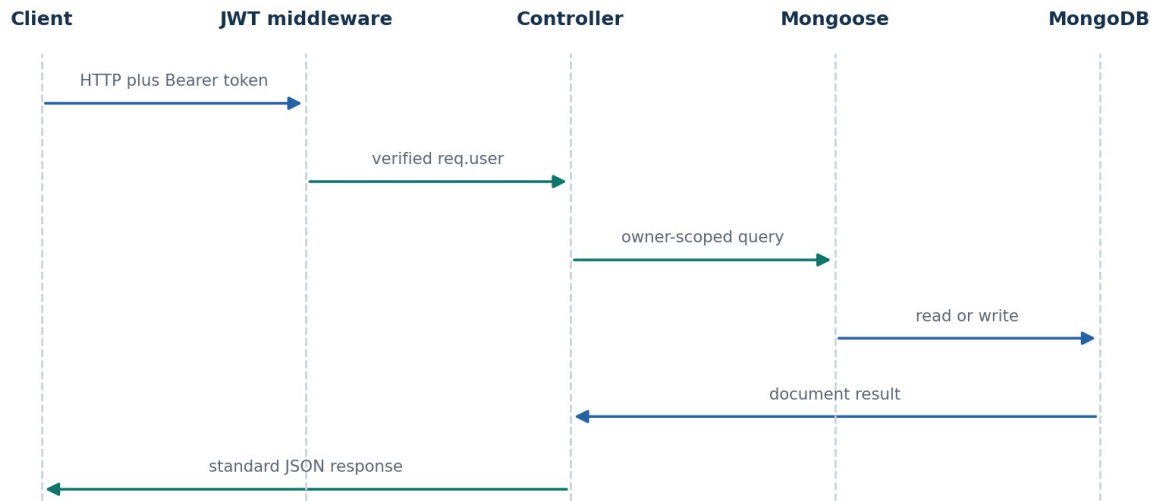


Figure 3 JWT verification and owner-scoped data access sequence

API contract catalog

Method	Endpoint	Access	Purpose	Main statuses
GET	/api/health	Public	Service health and runtime metadata	200
POST	/api/auth/register	Public	Create user and issue JWT	201, 400, 409

Method	Endpoint	Access	Purpose	Main statuses
POST	/api/auth/login	Public	Verify credentials and issue JWT	200, 400, 401
GET	/api/auth/email-available	Public	Check normalized email availability	200, 400
GET	/api/auth/profile	JWT	Return authenticated user profile	200, 401
POST	/api/workouts	JWT	Create an owner-scoped workout	201, 400, 401
GET	/api/workouts	JWT	List history and optional filters	200, 400, 401
GET	/api/workouts/search	JWT	Search by name, category or UTC date	200, 400, 401
GET	/api/workouts/:id	JWT	Read one owned workout	200, 400, 404
PUT	/api/workouts/:id	JWT	Update allowed workout fields	200, 400, 404
DELETE	/api/workouts/:id	JWT	Delete one owned workout	200, 400, 404
POST	/api/ai/workout-recommendation	JWT	Generate structured workout plan	200, 400, 429, 5xx
POST	/api/ai/fitness-insights	JWT	Generate submitted or database-derived insights	200, 400, 429, 5xx

Standard response envelopes

```

Success
{
  "success": true,
  "message": "Operation completed",
  "data": { "...": "endpoint-specific payload" }
}

Failure
{
  "success": false,
  "message": "Validation failed",
  "errors": [{ "field": "fieldName", "message": "Reason" }]
}

```

Security design

- Passwords are hashed in a Mongoose pre-save hook using bcrypt with a cost factor of 12.
- The password field is select:false and removed again during JSON and object serialization.
- JWTs carry the user id in sub, use issuer ai-fittrack-api and default to a one-day expiry unless configured.
- Protected middleware verifies the token, validates the ObjectId and reloads the user from MongoDB.
- Workout queries combine the requested identifier or filter with req.user._id.
- CORS allows configured origins, non-browser clients and the methods needed by the API.
- The JSON body limit is 1 MB and malformed JSON is converted to a 400 response.
- AI requests have a per-process default limit of 10 requests per user per minute.

Gemini integration design

The Gemini service builds constrained fitness prompts and requests JSON matching explicit schemas. Recommendation output includes a plan, weekly schedule, experience guidance, tips, recovery, safety and motivation. Insight output is based only on submitted metrics or the caller's owner-scoped records.

Control	Implemented behavior
Configuration	GEMINI_API_KEY is required; model, timeout and retry parameters are configurable.
Default model	gemini-3.6-flash in the uploaded source and supplied live evidence.
Timeout	30 seconds by default, capped at 120 seconds.
Retries	Default 3 attempts, maximum 5, exponential backoff, jitter and Retry-After support.
Response validation	Extract and parse a JSON object, then reject empty, malformed or incomplete responses.
Safe failure	Normalize missing key, bad key, unavailable model, rate limit, timeout and provider errors.

Environment configuration design

```
PORT=<local port>
MONGO_URI=<MongoDB connection string>
MONGO_USER=<optional separate username>
MONGO_PASSWORD=<optional separate password>
MONGO_AUTH_SOURCE=admin
JWT_SECRET=<strong private random secret>
JWT_EXPIRES_IN=1d
CLIENT_URL=http://localhost:5173
```

```
GEMINI_API_KEY=<private Google Gemini key>
GEMINI_MODEL=gemini-3.6-flash
GEMINI_TIMEOUT_MS=30000
GEMINI_MAX_ATTEMPTS=3
AI_RATE_LIMIT_PER_MINUTE=10
```

The values above are configuration names and safe placeholders only. The private .env file is intentionally absent from the submission.

Items requiring student confirmation

Pending item	What to provide
Individual identifiers	The four names and team designations are included. NM IDs and register or roll numbers were not supplied.
Institution details	College name, academic year, guide or mentor name and official submission date need confirmation.
Responsibility allocation	Team Leader and Team Member designations are recorded, but module ownership and task responsibilities were not supplied.
Repository	Insert the final public GitHub repository URL after upload.
Deployment	Insert the deployed API base URL if the evaluator requires a hosted demonstration.